

IRIS

TEAM CODE SMITHS

AI-enabled Low Cost Real Time Mine Subsidence
Monitoring, Prediction and Early Warning System
for Underground Coal Mines in India



CONTENTS

Problem, Positioning & Innovation	3
Scope: Prototype vs. Full-Scale	7
System Architecture	8
Sensing Layer (Three Tiers)	11
Firmware & Edge Algorithms	15
Dual-Mode Mining Physics	21
The Three-Layer / Three-Timescale Model	24
Digital Twin	25
Machine Learning	27
InSAR Pipeline	30
Backend, Database & Security	32
Frontend	37
Alerting & Emergency Path	42
Reliability & Naming Discipline	46
Feasibility, Impact & References	47

PART I — PROBLEM, POSITIONING & INNOVATION

1.1 The Problem

Underground coal mining causes gradual surface subsidence that threatens nearby communities, infrastructure, agricultural land, and forests. Indian subsidence monitoring today relies mostly on manual field surveys and after-the-fact damage assessment, which cannot give timely warning before ground failure occurs. By the time visible damage appears, the underlying deformation has already occurred. The goal is an indigenous, low-cost, real-time, AI-enabled early warning system using a wireless surface mesh sensor network deployed above underground mine panels.

1.2 Why This Is Genuinely Hard

- Subsidence develops gradually – there is no obvious single moment where "normal" becomes "critical."
- Ordinary mining activity looks like early subsidence. Blasting, heavy machinery, and transport cause ground vibration and minor tilt that resembles early subsidence signals but isn't dangerous – false alarms erode trust in the system.
- Real subsidence failure events are rare, so there is very little labelled data to train a conventional ML model on.
- Node placement matters as much as the sensors themselves – sensors in the wrong place simply won't see the signal in time.
- A single sensing modality cannot distinguish mechanism. Tilt alone can't tell you why the ground is changing, which matters for both accuracy and explainability to a mining engineer or DGMS reviewer.

1.3 Positioning Against Existing Solutions

Existing Approach	strengths	limitations	our solution
Worldsensing (global commercial)	Proven wireless mesh, tilt/piezometer/estensometer/GNSS, 10-year node battery life, long-range LoRa	Enterprise pricing – lakhs to crores per deployment	Same sensing concept, built on low-cost open hardware (ESP32/Arduino/Raspberry Pi)
CIMFR (Indian govt research, CSIR)	India-specific 3D subsidence prediction models for single- and multi-seam extraction	Research models, not a deployed real-time IoT product	Operationalise CIMFR-style geomechanical modelling into a live, continuous platform
InSAR / Sentinel-1 satellite	Free, wide-area ground deformation data, already validated against GPS in Indian coalfields (e.g. Raniganj)	6-12 day revisit cycle – cannot catch fast-developing failures in real time	Use satellite data to calibrate and enrich the model, never as the real-time safety layer

1.4 Where the Real Innovation Is

1. Hard-coding the mining influence function into the GNN's graph edge weights, rather than pure black-box learning or pure physics simulation – physics-constrained ML fusion, not seen applied to mine subsidence in the research reviewed.
2. The tiered evidence-fusion escalation pipeline (Kalman → Fuzzy Risk Index → Spatial Coherence → Evidence Fusion) as a deliberate, actively pressure-tested and corrected answer to the false-positive problem – most generic proposals don't address this at all.
3. A three-tier sensing design where each tier is explicitly mapped to a distinct, named subsidence mechanism (not "another way of sensing the same thing") – including a precise division of labour within Tier 2 itself (resistivity as direct detector, moisture as interpretation key, not two equally-weighted signals).
4. Dual-mode physics engine matching India's real mining mix – Longwall (PIM + Knothe) and Bord-and-Pillar (PHSR/CPHSR), selected per deployment. Over 90-95% of Indian underground coal production is Bord-and-Pillar; most comparable proposals implicitly target the minority (longwall) case.
5. Physics-Driven Digital Twin, not a 3D visualization gimmick – Monitoring Mode (continuous expected-vs-observed divergence) and Planning Mode (forward simulation of hypothetical future extraction).
6. Two independent, structurally dissimilar prediction engines (a learned PI-STGNN and a physics-only Digital Twin) combined through Risk Synthesis – deliberate redundancy so a blind spot in one can't silently take down the whole system.
7. Synthetic pretraining via the Probability Integral Method as a direct engineering answer to real failure-data scarcity, rather than assuming abundant labelled data.
8. Satellite InSAR as calibration, not decoration – real Sentinel-1 data used to calibrate physics parameters and cross-validate, honestly scoped around its 6-12 day revisit limitation.
9. Cloud-independent emergency alerting – when internet is down but cellular voice is up, a DTMF-authenticated call pathway can still trigger the siren mesh; the edge gateway can make and act on a local risk decision with zero cloud dependency at all.
10. Radical honesty in every output – "Fuzzy Risk Index" not "probability," satellite displacement labeled precisely, mock positions labeled as mock – built to survive real technical and regulatory scrutiny, not just a demo.

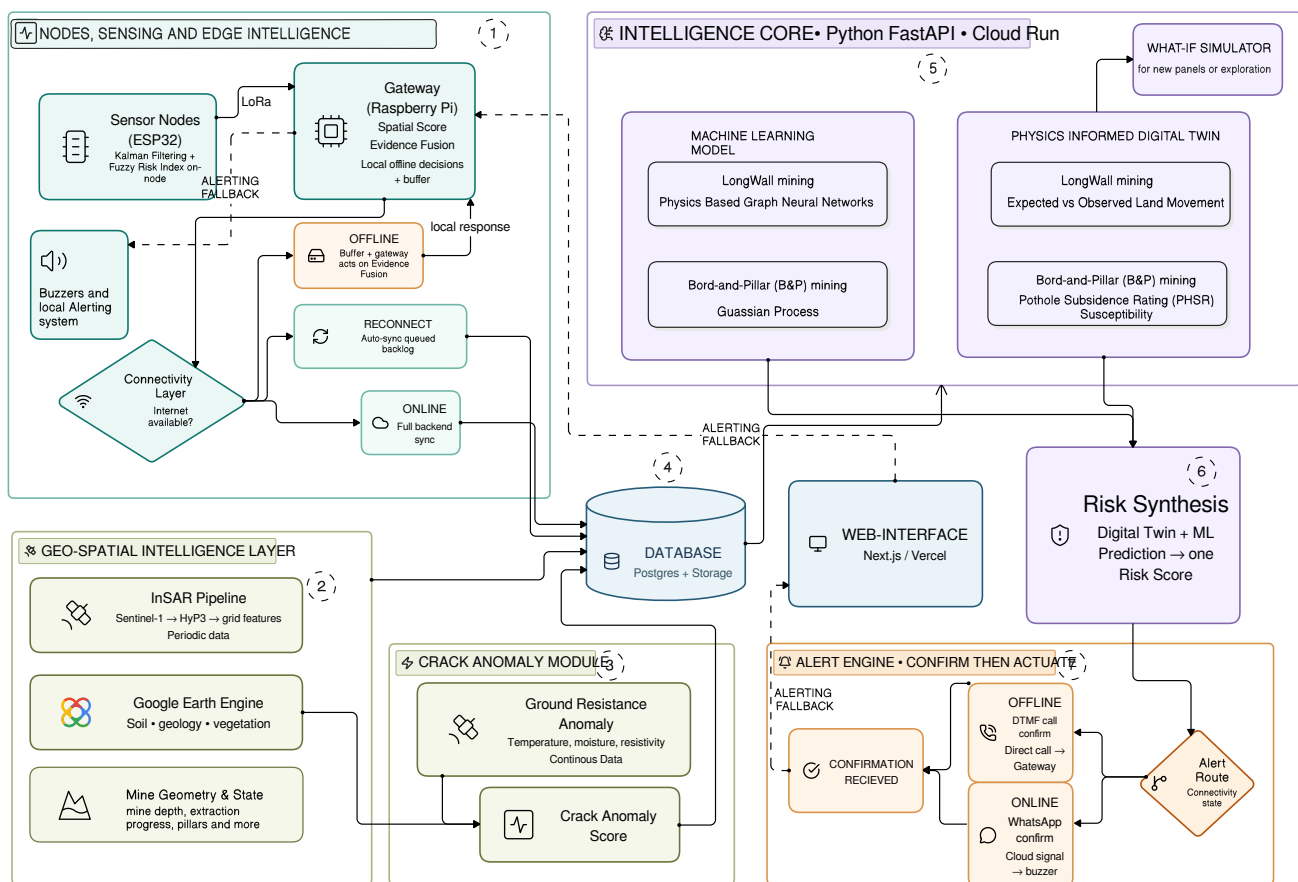
1.5 Root Causes of Subsidence and Mechanism Mapping

1. Goaf/void formation – the immediate cause: extracted coal leaves an underground cavity; the ground above eventually moves to fill that space.
2. Overburden/roof caving – rock layers directly above the goaf collapsing into it, propagating the void's effect upward.
3. Stress redistribution and fracturing – removing coal shifts load onto surrounding rock, which can fracture under the new stress pattern, opening pathways that didn't exist before. (This is the mechanism Tier 2 specifically targets.)
4. Delayed pillar failure (bord-and-pillar specifically) – pillars can weaken and fail years or decades after extraction, a distinct trigger/timeline from active longwall subsidence.
5. Groundwater/aquifer disruption – mining can drain or disturb water-bearing rock layers; loss of pore-pressure support can cause ground compaction through a mechanism distinct from cavity collapse. (Deliberately excluded from Tier 2's current scope – see Part 4.4.)
6. Rainfall/water infiltration – not a root cause on its own, but an accelerant: water softens rock, adds weight, changes pore pressure.
7. Time-dependent creep – some rock masses deform slowly under sustained stress even without a new trigger.
8. Multi-seam interaction – subsidence from one seam's extraction mechanically influencing a separately-mined neighbouring seam.

PART 2 — SCOPE: PROTOTYPE VS. FULL-SCALE

Aspect	Prototype (this build cycle)	Full-Scale (target design)
Sensor nodes	2 nodes, Tier 1 + Tier 2	Many nodes, geometry-informed density
Gateway	1 ESP32	Raspberry Pi, fog processing, offline buffering
Node-gateway radio	ESP-NOW, mesh topology	LoRa/Zigbee self-healing multi-hop mesh
Node positioning	GNSS one-time registration (real for physical nodes) + mock nodes for demo completeness	Same, at full deployment scale
InSAR	Real historical data (Raniganj), periodic batch ingestion	Same, live periodic ingestion tied to actual revisit cycle
Emergency alerting	Both online and offline paths demonstrated; offline path simulated for hardware, real for software logic	Full cellular voice hardware on gateway
Mining mode	Both Longwall and Bord-and-Pillar physics supported	Same, per-site selection
Heavy compute (GNN, physics twin)	Runs on laptop/Cloud Run, fed by gateway data	Runs in the cloud (same, at scale)
Power	USB / power banks	Solar + battery

PART 3 - SYSTEM ARCHITECTURE



3.1 End-to-End Data Flow

- Sensor Nodes (ESP32, Tier 1 + Tier 2)
- -> ESP-NOW (prototype) / LoRa+Zigbee mesh (full-scale) ->
- Gateway (ESP32 prototype / Raspberry Pi full-scale: Spatial Score + Evidence Fusion + offline decision logic)
- -> HTTPS POST /api/ingest ->
- Next.js API routes (the ONLY module touching Supabase/Python directly)
- -> writes ->
- Supabase (Postgres + Storage + Realtime, RLS-secured)
- <-> read/write <->
- Python ML/Twin Service (FastAPI, Google Cloud Run)
- - Digital Twin (Longwall + Bord-and-Pillar submodes))

- - PI-STGNN (Longwall) / Gaussian Process (Bord-and-Pillar)
- - Risk Synthesis (single fusion point)
- <-> reads InSAR + geology features <->
- InSAR Pipeline (Sentinel-1/HyP3) + Earth Engine (geology/vegetation)
- |
- v
- Next.js -> Website Dashboard
- |
- v
- Alert Engine -> ONLINE PATH (WhatsApp/SMS/Email confirm -> cloud signal -> gateway buzzer)
- > OFFLINE PATH (DTMF call confirm -> direct call -> gateway buzzer)

3.2 Layer Model

Layer	Contents	Rule
D — Presentation	Next.js App Router, React components, 3D twin	Consumes backend API only; computes nothing about risk
C — Application	Alert engine, notification engine, IVR orchestration	Adapters: ml-adapter, insar-adapter, gateway-adapter
B — API Boundary	Next.js route handlers, Zod validation, authn/authz	The only place other teams integrate with the backend
A — Persistence	Supabase Postgres, Row Level Security, Storage, Realtime	—

3.3 Module Ownership

Module	Responsibility	Interface exposed
Firmware (ESP32 + gateway)	On-node Kalman filtering, Fuzzy Risk Index, mesh comms, offline buffering	<code>reading</code> + <code>cluster_event</code> JSON payloads
InSAR pipeline	Sentinel-1 → ASF HyP3 → grid-cell displacement features	<code>insar_node_features</code> rows + rendered heatmap PNG
ML / GNN service	Physics-based GNN (Longwall) and Gaussian Process (Bord-and-Pillar) prediction	Fixed entry/exit contract; writes to <code>predictions</code>
Database	Schema, RLS, retention	Table schemas (owned by Backend team)
Backend API	Ingestion, validation, alerting, notification	All <code>/api/*</code> endpoints
Frontend + Digital Twin	Dashboard, visualization, 3D twin	Consumes backend API only
Alerting	Multi-channel escalation, IVR	Notification + call audit tables

3.4 The Adapter Pattern

Every external team's data passes through a dedicated adapter translating their raw shape into an internal domain type:

- `lib/adapters/ml-adapter.ts` -> `toPrediction(raw)`: `Prediction`
- `lib/adapters/insar-adapter.ts` -> `toInsarLayer(raw)`: `InsarLayer`
- `lib/adapters/gateway-adapter.ts` -> `toReading(raw)`: `Reading`

Each adapter also owns a mock implementation of the same type, selected by an environment flag (`ML_MODE=real|mock`). This decouples the frontend from upstream schema changes – a renamed field on the ML team's side requires changing one function, not every component that consumes it. It's also what allows the full dashboard to be built and demoed before every upstream service exists: `ml-adapter.ts` implements a three-tier fallback (live service -> latest stored row -> clearly-labeled mock with `is_stale: true`).

3.5 Why Not Connect Every Node Directly to the Cloud

- Power – a direct cellular/Wi-Fi radio draws far more power per transmission than short-range LoRa/Zigbee; running one on every node would drain batteries in days rather than years.
- Range/terrain – a mine panel can span a large area with obstructions; a single distant cloud connection is unlikely to reach every node reliably.
- Cost – a cellular SIM per node is far more expensive at scale than a handful of shared gateway units.
- Redundancy – with a mesh, neighbouring nodes route around a failed node; with direct-to-cloud, one node's radio failure just means that node goes dark with no recovery path.

PART 4 – SENSING LAYER (THREE TIERS)

4.1 Tier 1 – Kinematic/Deformation Sensing

- Question answered: Is the ground physically moving, and how fast?
Mechanism targeted: none specifically – detects the outcome of subsidence regardless of underlying cause; the universal, root-cause-agnostic layer every node carries.
- MEMS IMU (tilt + vibration), e.g. MPU6050, read over I2C.
- Displacement/strain sensing for local ground movement.
- Temperature sensing for MEMS thermal-drift compensation.
- Kalman-filtered on-node; raw residual also tracked as an independent detection pathway (Part 5).

4.2 Tier 2 – Crack/Ground-Anomaly Detection

- Question answered: Is a crack or fracture pathway forming at this location?
Mechanism targeted: stress-induced fracturing and the opening of a discrete pathway through the ground – a genuinely different phenomenon from Tier 1's bulk surface movement.
- Four components combined, not resistivity alone:
- Resistivity (primary, direct detector) – a 4-electrode AC-driven array. An open crack is physically an air gap in an otherwise electrically continuous soil/rock mass; air has far higher electrical resistance than soil, so a genuine

crack crossing the current path should produce a sharp, localised resistivity deviation (spike if dry, drop if water-filled) – a direct electrical consequence of the crack's geometry, not an inferred correlation.

- Moisture (interpretation key, not an independent detector) – a crack does not necessarily stay air-filled; if water enters (rain, mine drainage), the same crack can flip from high resistance (dry) to low resistance (water-filled, especially with dissolved minerals). The co-located capacitive soil moisture sensor doesn't itself claim "crack" or "no crack" – its job is to tell the fuzzy logic stage which direction of resistivity anomaly to expect right now, so the system correctly interprets whichever direction the anomaly moves rather than being confused by it. This is a deliberately more precise claim than treating resistivity and moisture as two independent, equally-weighted pieces of evidence – state this explicitly, it's a genuine design strength.
- Temperature – corrects MEMS/electrode thermal drift and provides context for interpreting resistivity alongside moisture.
- Geological context (Earth Engine: soil type, rock density) – establishes the expected baseline resistivity range for this specific ground type, so a measured value is compared against a geology-informed expectation rather than an arbitrary universal threshold.
- These four combine into a single Crack/Ground-Anomaly Score – interpretable, rule-derived, in the same naming spirit as the Fuzzy Risk Index, never a binary "crack detected" claim. Feeds the ML residual model as one additional input feature; explicitly excluded from the Digital Twin, whose defining boundary is taking no live sensor input at all.

Hardware:

Component	Role	Integration Note
4 stainless steel/copper electrodes	Resistivity electrode array	AC-driven (e.g. 555 timer astable circuit), not DC — DC causes electrolysis/corrosion and polarises readings over time
Known reference resistor	Voltage-divider with the soil path	Read as analog voltage on ESP32 ADC1
Capacitive soil moisture sensor	Moisture/interpretation-key reading	Capacitive, not resistive, to avoid corrosion within weeks; also on ADC1

4.3 Tier 3 – Reference Positioning and Satellite Calibration

Not a mechanism-sensing tier – it provides positioning and a wide-area reference/calibration layer alongside the local sensing tiers, never a third local mechanism in the same category as Tiers 1-2.

- GNSS-based node georeferencing – a one-time registration device (not continuous per-node GNSS) captures accurate real-world coordinates and a baseline sensor reading at installation, enabling spatial mapping, physics-based influence modelling, InSAR feature sampling, and PI-STGNN graph construction. `cumulative_displacement_since_baseline` is tracked as a first-class quantity, since Knothe's function is inherently cumulative and must be compared against a like quantity, not a raw instantaneous reading. A low-cost module (e.g. NEO-6M/NEO-M8N) gives ~2-5m accuracy – sufficient for georeferencing but not sufficient to directly detect millimetre-scale subsidence movement. RTK GNSS (centimetre-or-better) is a costed future upgrade, not adopted now.
- InSAR (Sentinel-1 via Copernicus/ASF HyP3) – used for calibration and validation, never the real-time trigger, honestly scoped around its 6-12 day revisit limitation. Feeds both mining modes as a supplementary evidence channel, joining at the fusion stage, never independently triggering.

4.4 Sensing Ideas Deliberately Considered and Deferred

Include this subsection – it demonstrates engineering maturity rather than a scattershot approach. All were explored in depth and found to have real scientific grounding, but deferred specifically for calibration/integration cost relative to the prototype's timeline, and to keep Tier 2 focused on one mechanism:

- Groundwater/aquifer sensing (water-table, pore-pressure) – targets a genuinely different mechanism family (mechanism 5 in Part 1.5) more central to large-scale groundwater extraction than to goaf-collapse-driven coal mine subsidence specifically.
- Thermal/void-proximity sensing – grounded in published sinkhole/void thermal-inertia research.
- Gas/pathway-migration sensing – grounded in soil-gas seepage research, with a dual-purpose tie to mandatory coal-mine methane safety monitoring.

- Passive self-potential sensing – a genuinely different, passive geoelectrical technique for detecting water movement through a pathway.
- Mechanical/optical direct crack-gap sensors (potentiometer extensometers, Time-of-Flight laser distance, Hall-effect/magnet pairs) – valid, buildable alternatives to electrical resistivity; remain available if resistivity+moisture proves insufficient on the demo rig, but resistivity was chosen as primary since it doesn't require a crack to already exist at a precisely pre-known point.

4.5 Geometry-Aware Node Placement

Sensor locations are determined using mine geometry, panel boundaries, angle-of-draw/influence zones, and reference areas outside the influence zone – never arbitrary spacing. Dense along the panel boundary/buffer ring (where strain concentrates first per subsidence theory), sparser interior, extra density at critical surface assets. Real-world node positions connect sensor observations to mine geometry, geological zones, influence relationships, and the Digital Twin – this geospatial anchoring is what makes every downstream physics calculation meaningful rather than abstract.

Full-scale deployment priority: active goaf edges -> zones with documented cracks/complaints/DGMS flags -> critical surface assets (villages, roads, water bodies) -> old bord-and-pillar workings (higher unpredictable long-term risk than active longwall panels).

PART 5 — FIRMWARE & EDGE ALGORITHMS

5.1 Where Each Algorithm Sits in the Flow

Kalman filtering runs on every reading, on the node, before transmission — never a gate, never discards anything. Logging rate is a separate, independent concern (dual-rate logging), so filtering and data retention are never conflated.

Raw sensor reading -> Kalman filter (every reading, on-node)

- > feature extraction (rate of change)
- > transmission to gateway
- > Fuzzy Risk Index (sensor-health-aware, on-node)
- > Spatial Score (continuous, gateway)
- > Evidence Fusion (gateway; combines all - no single stage can veto)
- > escalation decision
- > heavier prediction stage (event-triggered or scheduled background)

5.2 The Core Architectural Fix — Evidence Fusion Replaces Hard Gating

The risk identified: the pipeline was originally wired as a serial AND-chain — each stage (Kalman, fuzzy logic, spatial coherence) could unilaterally suppress a reading before the next stage ever saw it. A true, developing event could be killed early if any single cheap stage under-reacted to it, even if later confirmed by other evidence.

The fix: no single cheap algorithm retains permanent veto power. Kalman filtering, fuzzy logic, spatial coherence, and raw-residual monitoring now each produce independent evidence, combined into one weighted score. A sufficiently strong signal from any one channel alone can still escalate, even if others are inconclusive.

Other fixes from the same review:

- Raw residual monitoring — the residual between the raw reading and the current filtered estimate ($\text{raw} - \text{Kalman_estimate}$) is tracked as a second, independent detection pathway that doesn't depend on the smoothed signal at all. Fixes the risk that a conservatively-tuned Kalman filter (Q too small) could lag behind and smooth away a genuinely sudden real event.

- Sensor health as a distinct state, not risk = 0 – a faulty/unresponsive sensor now reports a distinct UNKNOWN risk state (a sentinel value), never a false "confirmed safe." Evidence fusion treats sensor health as a gate on trust, not on risk – the one place a hard gate remains appropriate.
- Output renamed for honesty – the fuzzy stage's output is explicitly the Fuzzy Risk Index throughout code, comments, and documentation, never a "probability" – a fuzzy output like 0.92 would otherwise invite an unjustified statistical claim.
- Spatial coherence made continuous, not binary – a strict binary "escalate only if a neighbour also looks elevated" rule could incorrectly suppress the very first node to detect a genuinely developing, still-localised event. Now returns a continuous 0.0-1.0 spatialScore() (proportion of fresh, elevated neighbours), one weighted input to evidence fusion rather than a gate – an isolated node can still escalate on the strength of its own evidence alone if strong enough.

5.3 Kalman Filter – Full Mechanics

Problem it solves: a true, slowly-changing quantity (e.g. actual ground tilt) can only be observed through a noisy sensor reading. The filter combines a prediction (based on the last estimate) with the new noisy reading, weighted by trust in each, to produce a better estimate than either alone.

State kept per sensor axis (two numbers): x – current best estimate of the true value; P – how uncertain that estimate is (a variance).

Predict step: $x_{\text{predicted}} = x_{\text{previous}}$ (ground tilt doesn't jump randomly). $P_{\text{predicted}} = P_{\text{previous}} + Q$, where Q is a small constant representing expected natural drift between readings.

Update step (when new raw reading z arrives): $K = P_{\text{predicted}} / (P_{\text{predicted}} + R)$ – the Kalman Gain (0 to 1), where R represents sensor noise (estimated empirically by holding the sensor still and measuring scatter). Then: $x_{\text{new}} = x_{\text{predicted}} + K * (z - x_{\text{predicted}})$, $P_{\text{new}} = (1 - K) * P_{\text{predicted}}$.

Intuition for K : noisy sensor relative to prediction confidence $\rightarrow$ K small, filter trusts its own prediction. Precise sensor $\rightarrow$ K close to 1, filter trusts the new reading. A self-adjusting weighted average, not fixed-ratio smoothing.

Worked example (true tilt steady at 0.25 degrees): raw readings 0.21, 0.31, 0.19, 0.28, 0.23, 0.34, 0.25 become filtered estimates approximately 0.21, 0.24, 0.22, 0.24, 0.24, 0.26, 0.26 – converges smoothly rather than tracking every noise spike.

Terminology clarification: "filter" here means "correct/clean every sample," never "remove some samples." Exactly one output estimate per input reading, always – nothing is ever dropped at this stage.

Honest limitation: if Q is set too small, the filter becomes overconfident and lags behind a genuinely sudden real event. If R is set too small, the filter over-trusts noisy raw readings. Mitigations: always log the raw reading alongside the filtered estimate (never only filtered); estimate R empirically per sensor before deployment; choose Q deliberately based on the fastest real event that must be tracked; document this as a known limitation rather than presenting the filter as infallible.

Final firmware code – `kalman_filter.h`:

```
#ifndef KALMAN_FILTER_H
#define KALMAN_FILTER_H

// Simple scalar Kalman filter - one instance per sensor axis
// (tilt_x, tilt_y, vibration, displacement each get their own instance).
//
// This filter never discards a reading: every call to update() consumes
// exactly one raw sample and returns exactly one filtered estimate.
// It only ever CLEANS data, it does not gate transmission or storage.
class KalmanFilter1D {
public:
    // Q: process noise - how much the true value is expected to drift
    // naturally between readings. Too small -> filter lags behind real
    // fast events. Tune this to the fastest real event you must track.
    // R: measurement noise - how noisy the raw sensor is. Estimate this
    // empirically: hold the sensor still, log readings, measure the
    // variance of that scatter.
    KalmanFilter1D(float q, float r, float initial_estimate = 0.0f)
        : Q(q), R(r), x(initial_estimate), P(1.0f) {}
};
```

```

// Call once per new raw reading. Returns the cleaned estimate.
float update(float raw_reading) {
    float x_predicted = x;
    float P_predicted = P + Q;
    float K = P_predicted / (P_predicted + R); // Kalman Gain, 0.1
    x = x_predicted + K * (raw_reading - x_predicted);
    P = (1.0f - K) * P_predicted;
    return x;
}

float getEstimate() const { return x; }
float getUncertainty() const { return P; }

// Raw residual: raw_reading - current smoothed estimate.
// A large, persistent residual is a SECOND, independent signal that a
// sudden real event may be occurring, even while the filtered estimate
// itself still looks moderate. Call this BEFORE update() with the new
// raw reading, using the estimate from the previous cycle.
float getResidual(float raw_reading) const { return raw_reading - x; }

void setNoiseParams(float q, float r) { Q = q; R = r; }

private:
    float Q; // process noise
    float R; // measurement noise
    float x; // current best estimate
    float P; // current estimate uncertainty (variance)
};
#endif

```

5.4 Fuzzy Logic – Full Mechanics

Problem it solves: avoid a hard cutoff (e.g. "tilt rate ≥ 0.3 deg/day = danger") that treats 0.29 and 0.31 as categorically different, and combine several differently-scaled/differently-meaning signals into one interpretable score.

Step A - Fuzzification: each input is converted into overlapping degrees of membership in named categories via triangular/trapezoidal shapes – e.g. LOW membership = 1.0 at 0.0 deg/day, dropping to 0.0 at 0.3 deg/day; MODERATE peaking at 1.0 at 0.3 deg/day; HIGH rising from 0.0 at 0.3 deg/day. A reading of 0.35 deg/day might come out as LOW=0.0, MODERATE=0.75, HIGH=0.25.

Step B - Rule evaluation: simple IF-THEN rules combine memberships across signals, e.g. "IF tilt_rate is HIGH AND vibration is SUSTAINED THEN risk is HIGH." AND = minimum of memberships; same-category-from-multiple-rules combine = maximum.

Step C - Defuzzification: the combined fuzzy result converts back to one crisp number via a weighted average across representative centre values, e.g. risk_score = $(\text{LOW} \cdot 0.15 + \text{MODERATE} \cdot 0.5 + \text{HIGH} \cdot 0.9) / (\text{sum of activations})$.

5.5 Spatial Coherence – Full Mechanics

At prototype scale (2-3 nodes): a direct comparison – if a node's risk score is elevated AND at least one physically nearby neighbour's risk score is also elevated within the same recent time window, escalate as spatially consistent; else suppress and log locally as an isolated, likely non-subsidence event. Deliberately the simplest correct case of a rule that generalises upward, not a hardcoded 2-node assumption.

At full-scale (dozens of nodes): generalises into a cellular-automaton propagation check – comparing observed risk-score patterns against propagation rules derived from angle-of-draw physics, since a genuine subsidence bowl should spread across neighbours in a physically consistent way an isolated fault or local disturbance would not.

5.6 Evidence Fusion – Full Mechanics

Runs on the gateway; replaces the old hard AND-gate. Combines Fuzzy Risk Index, spatial score, and raw-residual spike flag into one weighted evidence score, with an explicit rule that a sufficiently strong single signal can escalate on its own, and an explicit sensor-health gate.

5.7 Dual-Rate Logging (Solving the Data-Scarcity/False-Negative Risk)

A pure "only transmit/store when abnormal" design leaves no record of what "normal" looks like over time – no negative examples, no baseline to compare a suspicious period against. The adopted fix: dual-rate logging, applied independently of the escalation decision – every Kalman-filtered reading is always logged locally at a steady low rate (e.g. one summary per minute) as a continuous baseline, and whenever the fuzzy/spatial stages flag a concern, logging/transmission switches to a much higher rate for a window around the event. Both raw and filtered values, plus risk score and escalation flag, are stored together at all times – filtering never means discarding data, only ever choosing the rate of attention.

5.8 Hybrid GNN Triggering – Event-Gated + Scheduled Background

The gap identified and fixed: if the GNN only ever runs when fuzzy logic and spatial checks flag something, any case where those cheaper stages under-flag a genuinely developing but slow, borderline pattern means the GNN never gets a chance to catch exactly what it exists to catch.

The fix – hybrid triggering: (1) event-triggered runs whenever fuzzy logic and spatial coherence flag a concern (efficiency-driven path, unchanged), plus (2) a periodic low-frequency background run over the full recent stored window of data, regardless of whether anything was flagged – a safety net for slow, subtle patterns, at small bounded extra compute cost.

PART 6 — DUAL-MODE MINING PHYSICS

6.1 Why One Universal Subsidence Model Is Not Enough

India uses multiple underground mining configurations. Longwall mining systematically extracts a large section of coal; the roof is allowed to cave behind the advancing face, producing a relatively broad, progressive pattern of ground movement – a Knothe-based physics/empirical model can give an expected deformation pattern here ("at this location, the model expects approximately X mm of subsidence at this stage"), a physical quantity directly comparable to observations.

Bord-and-Pillar mining works differently – coal is partially extracted while pillars are left to support the roof. Depending on mining stage, geology, depth, extraction geometry, and pillar condition, instability can develop very differently – particularly during depillaring, when pillars are progressively removed. The resulting failure mechanism can be substantially more localized (pothole-style) than the broad subsidence trough of longwall mining. Using the same physical model for both would be a poor assumption.

Bord-and-Pillar represents over 90-95% of Indian underground coal production (85.76% mechanised + 0.10% conventional), vs. under 8% longwall (7.63% mechanised + 0.74% conventional) – most comparable proposals implicitly target the minority case. This is the single most defensible reason the dual-mode design matters – state it prominently.

6.2 Longwall Submode

Spatial formula:

$$W(x) = (W_0/2) * [\operatorname{erf}(\sqrt{\pi} * (x+D/2)/r) - \operatorname{erf}(\sqrt{\pi} * (x-D/2)/r)]$$

$$W_0 = a * m, \quad r = H / \tan(\beta)$$

Temporal formula (Knothe's Time Function):

$$W(t) = W_{\text{final}} * (1 - e^{(-c * (t-t_0))})$$

Where: a = subsidence factor (geology-informed prior), m = extraction thickness, H = depth, β = major influence angle, c = Knothe time coefficient,

tO = time extraction passed beneath that point – varies along an active advancing face, not one date for the whole panel; current_face_position/percent_complete in mine_geometry.json drives this directly. Inputs: mine_geometry.json (panel boundary, depth, extraction thickness, method, status, and for active panels: current face position/percent complete with timestamp); geology priors (a, beta, c) from Earth Engine (SoilGrids/OpenLandMap/HWSD) with SoilGrids' own published uncertainty propagated into confidence; calibration data (historical InSAR time series curve-fits a/beta/c when available, literature defaults otherwise); node baseline + cumulative displacement since installation; InSAR LOS displacement (fused as a side input). Outputs: expected deformation field (mm) per node/cell per time step; expected-vs-observed divergence (mm) – a direct physical comparison, valid because both sides are the same unit; Physics Deviation Index (0-1) = $|\text{observed} - \text{expected}| / \text{uncertainty_band}$; Planning Mode (simulated future field for a hypothetical extraction schedule, is_hypothetical: true, never written to live tables); multi-panel superposition (sum of independent panel fields – documented as a linear-independence simplification, not a true multi-seam model).

6.3 Bord-and-Pillar Submode

Three structural components, combined, none directly divergence-compared against sensors (this distinction from Longwall must be stated explicitly wherever B&P is discussed):

- 1.PHSR/CPHSR (Pothole Subsidence Rating) – an India-developed empirical framework for localized pothole-style subsidence, using rock-to-soil ratio, depth-to-height-of-extraction ratio, brittleness index, rock density. Semi-static – recomputed only when geology/geometry inputs change, not every sensor cycle. (Open item: obtain the actual CPHSR paper's scoring weights before finalizing the formula – currently using the identified parameter set, not verified coefficients – flag this honestly in the document.)
- 2.Tributary-area load + Obert-Duvall/CMRI strength -> Safety Factor, as a supporting factor within the same susceptibility layer:

CMRI has its own India-calibrated SF formula; Indian practice targets SF 1.5-1.8.

1. Deterministic (non-learned) cascading load-redistribution simulation – when a pillar's SF crosses below 1, its load redistributes to adjacent pillars, recomputing their SF. Graph-structured on pillar adjacency from mine geometry; explicit physics, no training, since this relationship is already exactly known and should not be relearned from scarce data.

Why PHSR is not directly compared to a sensor reading – worked example to include:

PHSR = 70, Tilt = 0.8 degrees. There is no scientifically meaningful statement like "PHSR minus tilt = 69.2." They are different kinds of quantities – PHSR represents structural susceptibility, tilt represents observed deformation. Instead: PHSR/CPHSR + current mining/structural state + processed sensor evidence + InSAR feed Risk Synthesis together, combining the different evidence streams rather than pretending they share a physical unit.

Critically: PHSR/CPHSR + current mining/structural state + processed sensor evidence + InSAR feed Risk Synthesis together as separate evidence streams.

Inputs: pillar/gallery geometry, depth, seam thickness; geology parameters (rock-to-soil ratio, brittleness, density); current mining/structural state (extraction %, pillar configuration, goaf development – manually updated for prototype).

Outputs: PHSR/CPHSR susceptibility score (semi-static, spatial); combined structural risk indicator that updates with mining-state changes even when PHSR itself hasn't; Planning Mode here is a susceptibility/risk projection under a hypothetical extraction percentage, NOT an mm-based deformation forecast – this asymmetry with Longwall's Planning Mode must be stated wherever shown.

Worked example – why PHSR alone isn't enough, and why mining-state matters:

A B&P region has PHSR = High. That tells us the area has high inherent susceptibility – it doesn't mean "collapse is happening right now." Now imagine mining state changes: before, extraction = 20%, pillars largely intact; later, extraction = 40%, several pillars removed, goaf expanded. Geological parameters haven't changed at all – PHSR = unchanged – but CURRENT STRUCTURAL STATE = changed. This is why PHSR should be treated as a baseline susceptibility layer, not the entire live risk calculation.

PART 7 — THE THREE-LAYER / THREE-TIMESCALE MODEL

Layer	Question Answered	Timescale	Example
1. Geological baseline	What is this location inherently like?	Very slow	Rock density, brittleness, rock/soil ratio
2. Mining/structural state	What is happening to the underground structure?	Changes with operations	Extraction %, pillar removal, goaf development, face position
3. Live observed response	What is the ground actually doing right now?	Live	Tilt, displacement, vibration, crack score, InSAR

Together, combined by Risk Synthesis, these give a materially richer picture than any single source alone – and this is exactly why PHSR can remain static while the system as a whole remains fully dynamic: the mine/structural state and live observation layers change continuously even when geology does not.

Information	Timescale	Example
Geological baseline	Very slow	Rock density, brittleness
Mine geometry	Slow / updated periodically	Depth, seam, pillars
PHSR/CPHSR	Semi-static	Structural susceptibility
Mining state	Changes with operations	Extraction %, depillaring
Digital Twin state	Continuously updated	Current mine representation
Sensor observations	Live	Tilt, cracks, vibration
Edge evidence	Near-real-time	Filtered anomalies
Risk synthesis	Near-real-time	Current risk state

PART 8 — DIGITAL TWIN

8.1 Naming Discipline

Never call this "digital twin" in code, schema, or API fields – internally it is the Physics Expected-State Model; "Digital Twin" is a pitch-level name only. The twin takes no live sensor data as input, ever – this is the boundary that makes it a genuinely independent, dissimilar check against the ML models, and lets it evaluate hypothetical future configurations with zero historical examples. It never grows a learned correction layer – parameter calibration via curve-fitting is "physics calibration," never "ML."

8.2 Structure by Submode

	Longwall Twin	Bord-and-Pillar Twin
Inputs	<code>mine_geometry.json</code> (incl. active face progress), geology priors, calibrated a/beta/c, node baseline/cumulative displacement, InSAR	Pillar/gallery geometry, depth, seam thickness, geology parameters, current mining/structural state, PHSR/CPHSR
Outputs	Expected deformation field (mm), expected-vs-observed divergence (mm, direct physical comparison), Physics Deviation Index (0-1)	PHSR/CPHSR susceptibility score, combined structural risk indicator (updates with mining state even when PHSR itself hasn't)
Planning Mode	Full mm-based simulated future deformation field for hypothetical extraction schedules	Susceptibility/risk projection under a hypothetical future extraction percentage — NOT an mm forecast; this asymmetry must be stated wherever shown

8.3 Validation Discipline

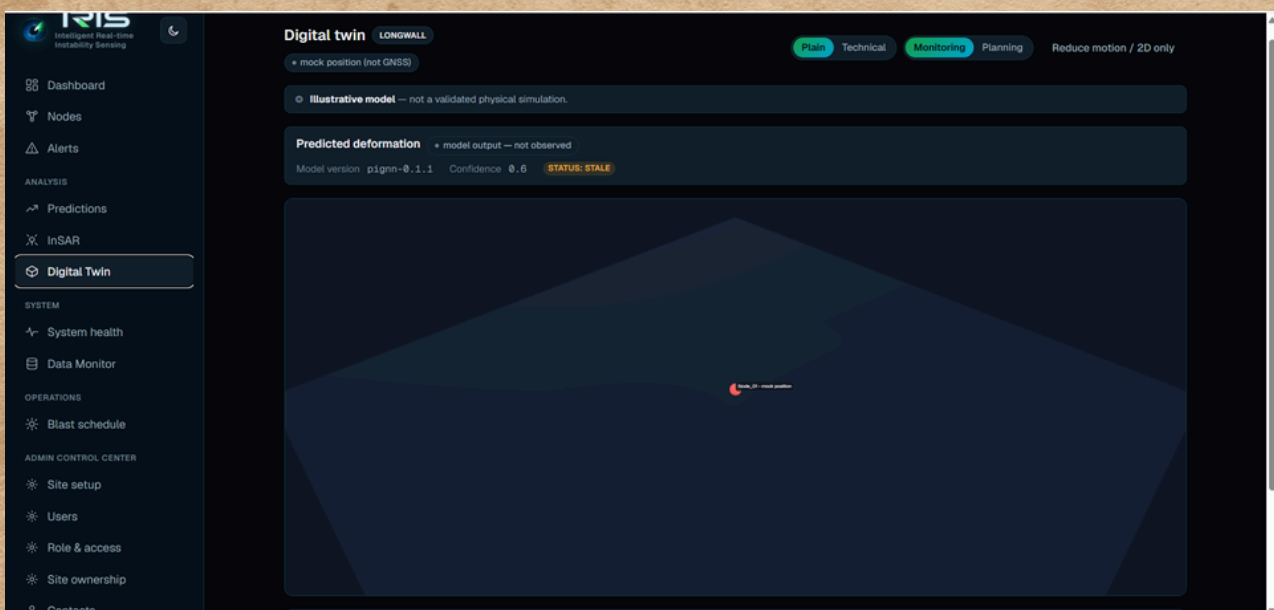
Run the residual calculator against synthetic data first, where expected and observed are equal by construction – a non-zero residual there indicates a bug in the residual math, not real ground behaviour. Honesty rule: a large residual is evidence, not proof – it may indicate genuine unexpected subsidence, or miscalibrated site parameters. Never presented as ground truth the sensors are checked against.

8.4 Why the Digital Twin Is Central

It is not simply a 3D model – it's the central representation of the mine's evolving state, containing geological information (rock type, density, brittleness, rock/soil ratio), geometric information (seam geometry, depth, extraction height, pillars, bords, mine boundaries), mining state (current extraction, sequence, pillar removal, goaf development), model outputs (PHSR/CPHSR, longwall expected subsidence), and live observations (sensor locations, tilt, displacement, crack trends, vibration, moisture, resistivity) – the place where "what the mine is" + "what mining has done to it" + "what the ground is doing now" are represented together.

8.5 Build Status Flag

Per the code-verified audit: the Bord-and-Pillar interaction slider is a real, tested, client-side illustration – moving the extraction-percentage slider shifts an illustrative risk indicator against a fixed PHSR-style baseline. It is not yet backed by a persisted `twin_state` table or a live trained model output. More broadly: no GNN, Gaussian Process, or PHSR model code was found in the audited repositories – the twin UI is a client-side illustration only. State this clearly rather than implying a working physics engine is live behind the visualization.



PART 9 — MACHINE LEARNING

9.1 PI-STGNN (Longwall)

Graph edge weights derived from angle-of-draw physics, not learned from scratch – the network learns only the residual deviation from known physics. This means the model doesn't need enough training examples to rediscover known mining physics; it only needs to learn the smaller residual patterns – the ways a specific real site deviates from idealised physics – a far easier learning problem given how little real failure data exists.

Frozen entry/exit contract:

ENTRY: time-windowed node readings, node positions, InSAR features

EXIT: {

```
  predicted_zone: [{node_id, severity_O_to_1}],  
  trend: "stable" | "accelerating" | "decelerating",  
  time_to_threshold: {low_days, high_days, confidence},  
  model_version
```

}

Pretrained on PIM/Knothe-generated synthetic data (deliberately imbalanced toward "nothing dangerous," matching real rarity); fine-tuned on real data. Triggered by gateway escalation OR a scheduled background pass this service owns (Vercel/Next.js cannot run persistent schedulers).

9.2 Gaussian Process (Bord-and-Pillar)

Chosen over a GNN specifically because it fits a small-real-data, local/per-zone residual task better, and natively outputs an uncertainty band. Tracks the residual between the PHSR/CPHSR+mining-state expected condition and live sensor/InSAR observations over time at each zone – not the cascading load-transfer mechanism, which stays deterministic (explicit physics already known, not something to relearn). Output feeds Fukuzono-style extrapolation (trend + uncertainty), the same underlying method as Longwall's inverse-velocity step, applied to a different quantity.

9.3 Fukuzono's Inverse-Velocity Method (post-processing, both modes)

What it is: an established method from landslide early-warning science for converting an accelerating deformation trend into an estimated time-to-failure.

How it works: as ground approaches structural failure, velocity doesn't increase linearly – it accelerates such that the inverse of velocity ($1/\text{velocity}$) decreases in an approximately straight line toward zero as failure approaches. At each time step, compute predicted deformation velocity, take its inverse, plot over recent time steps, fit a straight line, extrapolate forward to find where it crosses zero – that crossing point is the estimated time of failure (inverse velocity of zero = theoretically infinite velocity = sudden collapse).

Uncertainty/confidence: the inverse-velocity extrapolation is run several times using slightly different variations of the model's output (an ensemble – e.g. by slightly perturbing the recent input window). The spread of resulting time-to-failure estimates converts into a confidence range (e.g. "12-18 days, 75% confidence") instead of one falsely-precise number. `time_to_threshold` is always shown as a range, never a single date.

9.4 Risk Synthesis (single cloud-side fusion point)

Combines exactly two named O-1 inputs per active mode:

- Longwall: GNN Severity + Physics Deviation Index
- Bord-and-Pillar: GP residual output + PHSR/CPHSR-derived structural risk

Using the same no-single-channel-vetoes evidence-fusion philosophy already used at the gateway. Outputs one Risk Score (Normal/Watch/Warning/Critical), one recommended action, one time-to-threshold window, per panel and per node. A separate confidence badge (model agreement) is computed alongside but never shares the risk color scale

9.5 Sim-to-Real Transfer and Synthetic Data Generation

The model is first pretrained entirely on synthetic data generated by the Probability Integral Method + Knothe's Time Function across many randomised panel geometries and calibrated parameters. Once deployed at a real site, it continues training ("fine-tuning") on real sensor data collected there, using a smaller adjustment step size so it refines its understanding to the specific site

rather than discarding the broader synthetic-learned knowledge. Dataset schema: `scenario_id`, `source` ("synthetic"|"real", never silently mixed), `node_positions`, `panel_geometry`, `time_series` (per-node per-timestep filtered readings + `fuzzy_risk_index`), `label` (present only for synthetic or confirmed-real scenarios). Class imbalance is constructed deliberately (most scenarios = nothing dangerous), matching real-world rarity.

9.6 Open Decision – 2-Node vs. Larger Synthetic Topology

Unresolved at time of writing: pretrain/fine-tune strictly on 2-node graphs (matching the prototype exactly, but giving the GNN little spatial structure to learn from), or pretrain on larger synthetic node arrays matching the full-scale target topology, then fine-tune/evaluate on the 2-node real subgraph. Must be decided before the GNN training stage.

PART 10 — INSAR PIPELINE

10.1 Real, Operating Pipeline

Sentinel-1 discovery -> pair selection/validation -> HyP3 submission/monitoring -> grid spatial feature extraction -> temporal QC and feature engineering -> validated dataset. Real processed output exists over Shyamsundarpur UG Mine, Raniganj Coalfield – a real, published room-and-pillar InSAR precedent, strengthening both the twin's and the pitch's real-world grounding. Production pipeline status: Done – live-imported, 1,591 grid cells x 6 date-pairs.

10.2 Naming/Honesty Discipline

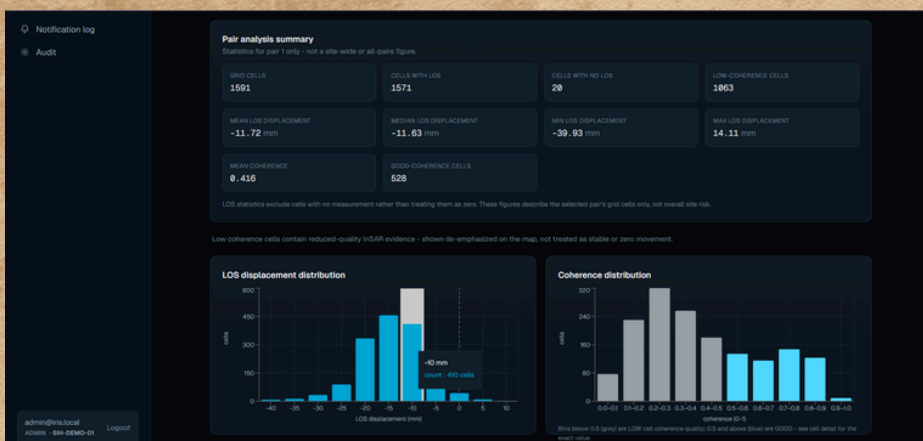
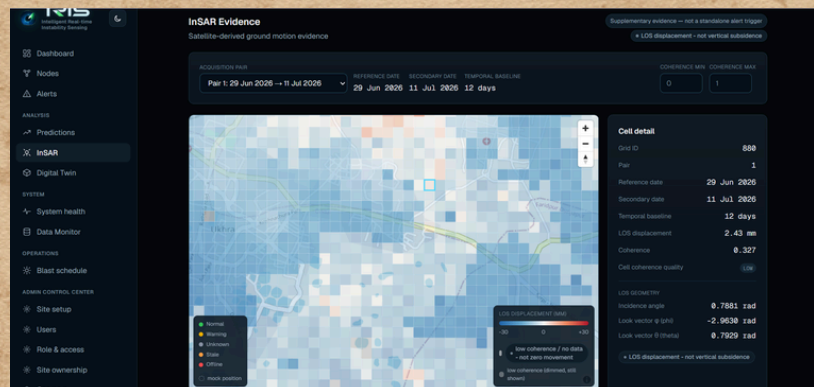
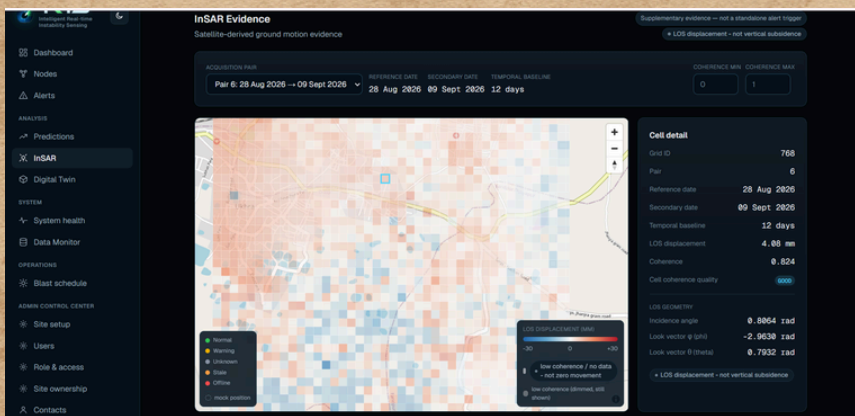
- LOS displacement is stored in millimetres; unit conversion from the pipeline's native metre output is an explicit, named step, never implicit.
- A single scene pair yields displacement between two dates, never an annualized velocity, unless genuinely derived from a multi-date time series. Field is named `insar_los_displacement_mm`, deliberately corrected from an earlier draft's `insar_los_velocity_mm`, which risked implying an annualized rate from a single pair.
- Displacement is always relative to a stated reference point, never absolute.
- Low coherence means unreliable/unavailable, never zero movement; coherence threshold is a documented, tunable parameter, not a universal constant.
- Feeds both mining modes as a supplementary evidence channel, joining at Risk Synthesis, never independently triggering an alert.
- Node-to-grid-cell spatial join (nearest-cell lookup from each node's GNSS-registered coordinate) is owned by the ML/Twin module.

10.3 Final Deliverable Structure

- `FINAL_INSAR/raw_metadata/`
- `FINAL_INSAR/processed/{los_displacement, coherence, phase}/`
- `FINAL_INSAR/spatial/insar_grid_features.csv`
- `FINAL_INSAR/temporal/insar_timeseries.csv`
- `FINAL_INSAR/metadata/insar_metadata.json`
- `FINAL_INSAR/qc/quality_report.csv`
- `FINAL_INSAR/README.md`

10.4 Unresolved Integration Item (surface prominently, this is a real blocker)

The InSAR module keys its output by cell_id (a grid cell); the ML pipeline and node schema key by node_id (a physical/mock sensor location). A grid cell is a spatial analysis unit, not a physical sensor – the node-to-cell spatial join is not yet owned by any module and must be assigned before the ML pipeline's feature engineering stage can correctly consume InSAR features per node. Also required: the ML service must consume the InSAR module's finished, QC'd, reference-corrected CSV outputs – it must not independently re-implement raw GeoTIFF sampling or coherence filtering, which would risk two inconsistent implementations of the same correction logic.



PART II — BACKEND, DATABASE & SECURITY

11.1 API Conventions

- Base path /api.
- All success responses: { "data": ..., "meta": { "generated_at": ..., "source": "live|mock|cached" } }.
- All errors: { "error": { "code": ..., "message": ..., "details": [...] } } with correct HTTP status – never a bare 500 for a malformed client payload.
- Every value with a unit carries the unit string.
- Every timestamp is ISO 8601 UTC; the frontend converts to IST for display.

11.2 Ingestion Endpoint

POST /api/ingest – machine-to-machine, authenticated via shared-secret header (x-gateway-key) or HMAC-SHA256, deliberately separate from user RBAC. Pipeline: authenticate -> rate-limit per node -> Zod-validate against schema_version -> normalize units -> dedupe on (node_id, seq_num) -> insert -> trace -> if cluster_event.escalate, invoke the alert engine synchronously as an internal function call. Responses: 202 accepted, 200 on detected replay/duplicate, 400 validation error with field paths, 401 bad auth, 429 rate-limited.

11.4 Database Design Principles

1. Core tables (readings, cluster_events, insar_node_features, predictions) are shape-locked since other teams write to/read from them directly.
2. Nine application tables (nodes, alerts, contacts, notifications, call_sessions, alert_feedback, blast_schedule, pipeline_trace, audit_log, sites) are owned by the backend team to fill gaps the core schema leaves open.
3. UTC everywhere in storage (timestampz); IST only at display time.
4. Two timestamps per ingested row – recorded_at (gateway clock) and received_at (server clock) – to compute latency honestly under clock skew.
5. Unknown is a first-class state. A sensor-health failure is never coerced to NULL->O->"safe."

11.5 Table Inventory

Table	Owner	Written by	Read by
<code>sites</code>	Backend	Backend (seed)	All
<code>nodes</code>	Backend	Backend (seed/admin)	Frontend, ML, InSAR
<code>readings</code>	Shared spec	Gateway (via ingest)	ML, dashboard
<code>cluster_events</code>	Shared spec	Gateway (via ingest)	Alert engine
<code>insar_node_features</code>	Shared spec	InSAR team	Alert enrichment, dashboard
<code>predictions</code>	Shared spec	ML team	Dashboard, twin, alert enrichment
<code>alerts</code>	Backend	Alert engine	Dashboard, notifications
<code>contacts</code>	Backend	Admin UI	Notification engine
<code>notifications</code>	Backend	Notification engine	Dashboard audit
<code>call_sessions</code>	Backend	Voice adapter + webhooks	Dashboard audit
<code>alert_feedback</code>	Backend	IVR webhook / dashboard	ML team (labelled data)
<code>blast_schedule</code>	Backend	Admin UI	Alert engine
<code>pipeline_trace</code>	Backend	Every pipeline stage	Incident timeline view
<code>audit_log</code>	Backend	Auth-guarded mutations	Compliance view

11.6 Notable Table – nodes

Without a locked schema here, different teams independently invent conflicting node identifiers (NODE_01, node01, N01, 1):

```
create table nodes (  
  node_id int primary key,  
  site_id text not null references sites(site_id),  
  label text not null,  
  mock_latitude double precision not null,  
  mock_longitude double precision not null,  
  is_mock boolean not null default true,  
  elevation_m double precision,  
  installed_at timestamptz,  
  is_active boolean not null default true,  
  created_at timestamptz not null default now(),  
  updated_at timestamptz not null default now()  
);
```

11.7 Notable Table – alerts and Its Lifecycle

```
create type alert_severity as enum ('info','warning','high','critical');  
create          type          alert_source          as          enum  
('cluster_event','prediction','manual','system_health');  
create          type          alert_state          as          enum  
('new','notified','acknowledged','resolved','dismissed');  
  
create table alerts (  
  id bigint generated always as identity primary key,  
  site_id text not null,  
  node_id int references nodes(node_id),  
  severity alert_severity not null,  
  source alert_source not null,  
  state alert_state not null default 'new',  
  cluster_event_id bigint references cluster_events(id),  
  prediction_id bigint references predictions(id),  
  evidence_score real,  
  reason text not null,
```

```

reason text not null,
summary text not null,
evidence jsonb not null default '{}::jsonb,
correlation_key text not null,
event_count int not null default 1,
first_event_at timestamptz not null,
last_event_at timestamptz not null,
blast_suspected boolean not null default false,
blast_schedule_id bigint references blast_schedule(id),
created_at timestamptz not null default now(),
notified_at timestamptz,
acknowledged_at timestamptz,
acknowledged_channel text,
resolved_at timestamptz,
resolution_note text
);

```

```

create unique index alerts_open_correlation_idx
on alerts (correlation_key) where state in ('new','notified');

```

11.8 Row Level Security Summary

Table	anon (gateway)	authenticated (operator)	service_role
<code>readings</code> , <code>cluster_events</code>	INSERT only	SELECT	ALL
<code>nodes</code> , <code>sites</code> , <code>predictions</code> , <code>insar_node_features</code>	none	SELECT	ALL
<code>alerts</code>	none	SELECT + UPDATE (state fields)	ALL
<code>contacts</code>	none	SELECT only if <code>role=admin</code>	ALL
<code>blast_schedule</code>	none	SELECT; INSERT/UPDATE if <code>role=operator</code>	ALL

11.11 SensorDataPacket Schema (Firmware <-> Gateway, ESP-NOW)

```
typedef struct __attribute__((packed)) {
    uint8_t msg_type;      // MSG_TYPE_SENSOR_DATA
    uint8_t schema_version; // bump on any field change
    uint8_t source_node_id; // originating node, unchanged through relays
    uint8_t hop_count;      // 0 = direct; incremented by each relay hop
    uint32_t seq_num;       // increments every packet from this node
    uint32_t timestamp_ms;  // node's own millis(), NOT wall-clock
    float tilt_x_raw;       // degrees
    float tilt_y_raw;
    float tilt_x_filt;      // Kalman output
    float tilt_y_filt;
    float vibration_raw;    // unit TBD from hardware team
    float vibration_filt;
    float displacement_raw; // mm
    float displacement_filt;
    float risk_score;       // computed ON-NODE by fuzzy logic, 0.0-1.0
    uint8_t node_status;    // bit flags below
} SensorDataPacket; // 47 bytes total
```

Bit	Flag	Meaning (1 = set)
0	STATUS_SENSOR_OK	Sensor is healthy
1	STATUS_LOW_BATTERY	Battery below defined threshold
2	STATUS_SELF_TEST_FAIL	Most recent self-test failed
3	STATUS_COMM_QUALITY_LOW	Weak/degraded radio link
4	STATUS_CALIBRATION_STALE	Overdue recalibration
5-7	reserved	Default 0

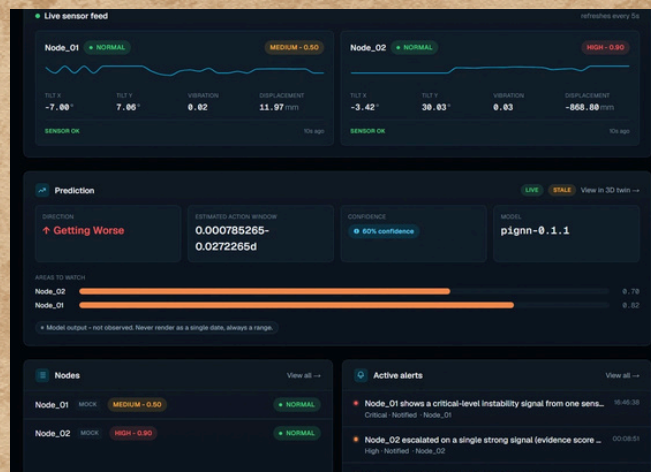
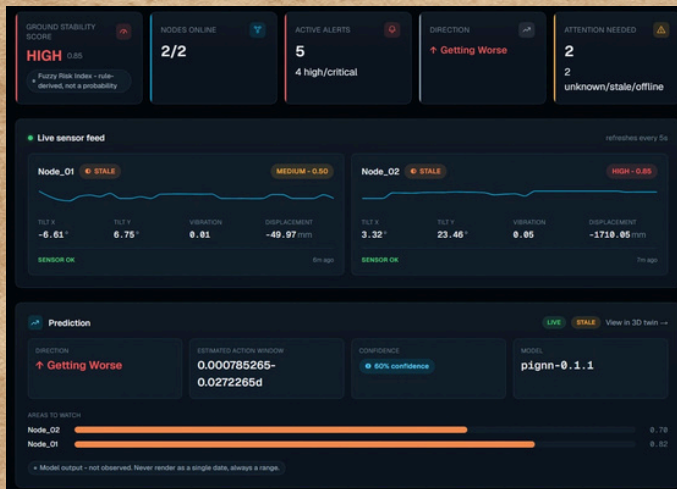
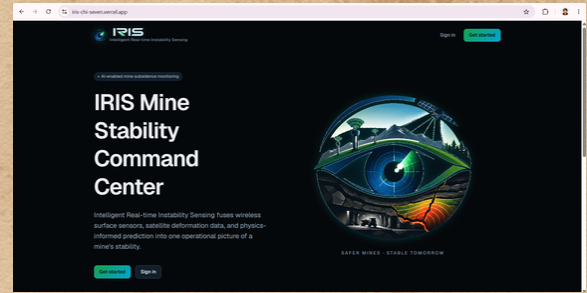
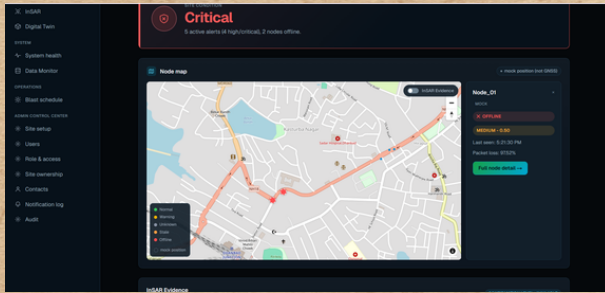
PART 12 — FRONTEND

12.1 Route Map

Route	Purpose
<code>/dashboard</code>	Command Center — status strip, map, active alerts (Plain View default)
<code>/nodes</code>	Node list + health table
<code>/nodes/[id]</code>	Node detail — live values, charts, InSAR sample, history
<code>/alerts</code>	Active / Acknowledged / Resolved / All
<code>/alerts/[id]</code>	Incident view + pipeline timeline + evidence + actions
<code>/twin</code>	3D digital twin (full screen)
<code>/insar</code>	InSAR layer analysis + metadata + caveat panel
<code>/predictions</code>	Latest prediction, predicted zone, time-to-threshold, model version
<code>/admin/*</code>	Contacts, blast schedule, notification/call audit, roles (admin-only)
<code>/system</code>	Gateway, node, ML, InSAR, DB, provider health
<code>/settings</code>	Thresholds, suppression window, units, demo mode toggle

2.2 Design Principles

- Plain View is the default landing state — one hero risk number (Normal/Watch/Warning/Critical), not a stat-card wall, aimed at a safety officer rather than an engineer.
- Confidence is a badge visually separate from risk color — model disagreement is never confused with danger level.
- Fixed status vocabulary used identically everywhere: NORMAL - LOW - MODERATE - HIGH - CRITICAL - UNKNOWN - OFFLINE - STALE.
- Risk state is never encoded in color alone — always color + icon + text.
- Non-negotiable label components, built as reusable components rather than strings so they cannot be forgotten: MockPositionLabel, LosDisplacementLabel, FuzzyIndexLabel, SinglePairLabel, NoDataLegend.
- All times shown in IST with an explicit suffix; hover reveals UTC.



12.3 Plain View / Technical View Translation Table

Technical term	Plain-view label
Fuzzy Risk Index	"Ground Stability Score"
Evidence Score	"Confirmation Level"
<code>time_to_threshold</code>	"Estimated Action Window"

12.4 Visualization Layers Table

Layer	Content	Mandatory framing
Sensor risk	Per-node markers colored by Fuzzy Risk Index, plus a physics-derived angle-of-draw influence radius	Any interpolated surface between sparse nodes is labeled "interpolated — illustrative only"
InSAR	Pre-rendered PNG draped with correct bbox/CRS	Titled "InSAR LOS displacement," never "vertical subsidence"; acquisition dates shown; low-coherence pixels transparent, never green/zero; explicit note that InSAR is supplementary and never independently triggers an alert
Prediction	Severity blobs / choropleth, terrain displacement in 3D	<code>time_to_threshold</code> always shown as a range, never a single date

12.5 Setup / Onboarding Flow (runs once per deployed site)

Populates `site_config` plus the shared `mine-geometry.json`-equivalent record. Every field must be explicitly tagged `is_assumed: true/false` in storage – this flows through to every dashboard display that uses it. Six steps:

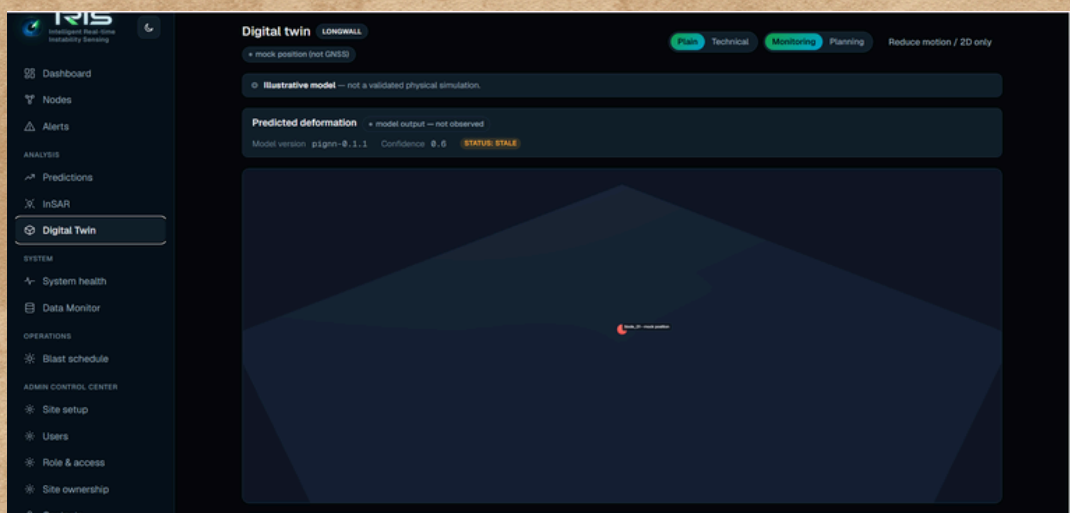
1. Mine Identity – name, real-world AOI coordinates (anchor to a real location even if layout is illustrative, since Earth Engine/InSAR data is only meaningful over real coordinates); `mine_type` (Longwall | Bord-and-Pillar) – the master switch determining downstream forms and physics/ML submodules.
2. Geometry Inputs – mode-specific (Longwall: panel boundary, depth, extraction thickness, status, active-face details; B&P: pillar width, gallery/bord width, depth, seam thickness); geology parameters with an explicit "Use Earth Engine estimate" vs. "Enter known value" toggle per field.

1. Node Registration – node ID, GNSS-registered coordinates (one-time capture at install), installation timestamp, baseline sensor reading at $t=0$, source: "real"|"mock" flag; support for explicit mock nodes for demo completeness, visibly flagged everywhere downstream.
2. Data Source Configuration – InSAR AOI confirmation + data freshness display; Earth Engine live connection status with a "use static geology fixture" fallback toggle.
3. Emergency Contacts / Escalation Chain – ordered contact list with channel preferences per contact for the normal-connectivity path.
4. Review and Confirm – full summary screen showing every entered value with its assumed/real flag, before writing to site_config and unlocking the main dashboard.

12.6 The 3D Digital Twin (Visualization Layer)

Stack: react-three-fiber + drei + three. Scene: subdivided terrain plane, extruded underground mine-panel geometry, instanced node-pin meshes colored by risk, an InSAR heatmap texture toggle, a time scrubber (Past <- Now -> Prediction). Signature feature: terrain-vertex displacement driven by the predicted subsidence profile, rendering the predicted trough directly rather than only marking risk with colored pins. Honesty requirement: the deformed surface is explicitly labeled "Predicted deformation (model output – not observed)" with model_version and confidence shown, since no component in the system has been empirically validated against real failure data.

Distinct from the compute-side Physics Expected-State Model (Part 8) – this is purely a consumer of /api/predictions/latest, /api/twin/state, and /api/twin/whatif; never a data source, never computing anything the backend hasn't already provided.



12.7 Data Layer

TanStack Query for all reads (caching, background refetch, retry with backoff); realtime/SSE updates invalidate query keys rather than mutating state directly; Zod-inferred types shared between API and client so both sides cannot silently drift apart. Documented exception: the frontend may subscribe directly to Supabase Realtime for live reading push-updates – the one exception to "frontend never touches Supabase directly."

The screenshot shows the IRIS Data Monitor interface. The left sidebar contains navigation links: Dashboard, Nodes, Alerts, ANALYSIS (Predictions, InSAR, Digital Twin), SYSTEM (System health, Data Monitor), OPERATIONS (Blast schedule), and ADMIN CONTROL CENTER (Site setup, Users, Role & access, Site ownership, Contacts, Notification log). The main content area is titled "IRIS Data Monitor" and "Live operational data from the IRIS backend - internal, for development and demonstration." It displays a grid of 18 data cards, each with a title, status, description, and row count. The cards are: Overview, Nodes (OPERATIONAL, ~2 rows), Readings (OPERATIONAL, ~371 rows), Cluster Events (OPERATIONAL, ~151 rows), Alerts (OPERATIONAL, ~7 rows), Predictions (OPERATIONAL, ~119 rows), InSAR Node Features (OPERATIONAL, ~0 rows), InSAR Grid Cells (OPERATIONAL, ~1591 rows), InSAR Grid Observations (OPERATIONAL, ~9546 rows), Blast Schedule (OPERATIONAL, ~1 rows), Pipeline Trace (SYSTEM, ~2426 rows), Alert Feedback (OPERATIONAL, ~0 rows), Site Config (SAFE INTERNAL, ~0 rows), Notifications (ADMIN / PI, ~14 rows), Contacts (ADMIN / PI, ~1 rows), Users / Profiles (ADMIN / PI, ~2 rows), and Audit Log (SYSTEM, ~12 rows).

This screenshot shows a different view of the IRIS Data Monitor interface, focusing on the "InSAR Grid Cells" and "InSAR Grid Observations" sections. The left sidebar is identical to the previous screenshot. The main content area displays a grid of 18 data cards, each with a title, status, description, and row count. The cards are: Predictions (no rows), InSAR Node Features (no rows), InSAR Grid Cells (OPERATIONAL, ~1591 rows), InSAR Grid Observations (OPERATIONAL, ~9546 rows), Blast Schedule (OPERATIONAL, ~1 rows), Pipeline Trace (SYSTEM, ~2426 rows), Alert Feedback (OPERATIONAL, ~0 rows), Site Config (SAFE INTERNAL, ~0 rows), Notifications (ADMIN / PI, ~14 rows), Contacts (ADMIN / PI, ~1 rows), Users / Profiles (ADMIN / PI, ~2 rows), and Audit Log (SYSTEM, ~12 rows).

PART 13 — ALERTING & EMERGENCY PATH

13.1 Governing Rule

Both online and offline paths share the same structure and the same governing rule: the complete detection pipeline (through ML/Twin/Risk Synthesis) must already have reached a confirmed critical state before either path activates, and a human executive must confirm before sirens actuate – only the communication and confirmation CHANNEL differs, never the decision authority.

13.2 Online Path (Internet Available)

Risk Synthesis: CONFIRMED CRITICAL

|

v

WhatsApp message to executive (Twilio WhatsApp Business Platform)

-- structured summary: zone, risk level, trend, time-to-threshold

|

v

Executive confirms/authorizes (WhatsApp reply or in-app confirmation)

|

v

Cloud sends an ONLINE SIGNAL to the gateway (HTTPS/MQTT push)

|

v

Gateway triggers local siren mesh; per-node activation acknowledgement

13.3 Offline Path (Internet Down, Cellular Voice Available)

Risk Synthesis: CONFIRMED CRITICAL (computed before connectivity loss,
or via the gateway's own local Evidence Fusion decision if fully offline)

|

v

AI-generated voice call to executive (Twilio Programmable Voice, TTS)

-- structured summary spoken aloud, DTMF prompt: "Press 9 to authorize"

|

v

Executive confirms via DTMF keypress (not voice recognition –
deliberate, for reliability under safety-critical conditions)

|

v

A SECOND call is placed directly to the gateway's own cellular line

|

v

Gateway auto-answers, validates an authenticated DTMF command
(event ID + one-time token + caller-ID whitelist as a second check)

|

v

Gateway triggers local siren mesh; per-node activation acknowledgement

13.4 DTMF Digit Meanings

Digit	Verdict	System behaviour
1	<code>real_event</code>	Alert acknowledged; remaining contacts informed as informational
2	<code>blast_or_disturbance</code>	Alert acknowledged, <code>blast_suspected=true</code> , notifications suppressed for a bounded, admin-configurable window (default 30 min)
3	<code>uncertain</code>	Alert acknowledged but escalation ladder continues
9	—	Replay message

13.5 Escalation Ladder Timing (Normal Path)

alert created

-> t+0s dashboard banner + realtime push (all operators; never suppressed)

-> t+0s email -> every active contact for the site

-> t+30s SMS -> priority-1 contact

-> t+60s voice call -> priority-1

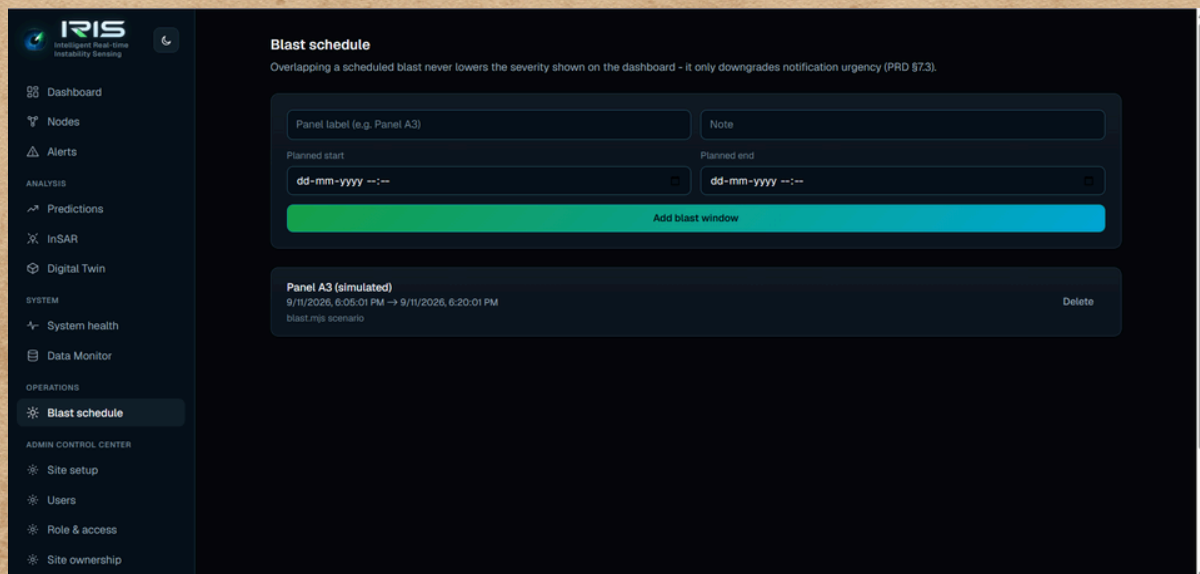
-- answered + DTMF -> record feedback, stop ladder

-- no answer/busy -> t+150s voice call -> priority-2

-- still nothing -> t+240s voice call -> priority-3

13.6 Blast-Context Enrichment

A `blast_schedule` overlap on the alert time (+/- configurable margin) sets `blast_suspected = true`, downgrades notification urgency by one channel step (voice -> SMS), but never downgrades the severity shown on the dashboard and never suppresses logging, ingestion, or ML processing. This is the system's answer to false alarms caused by routine, scheduled blasting activity.

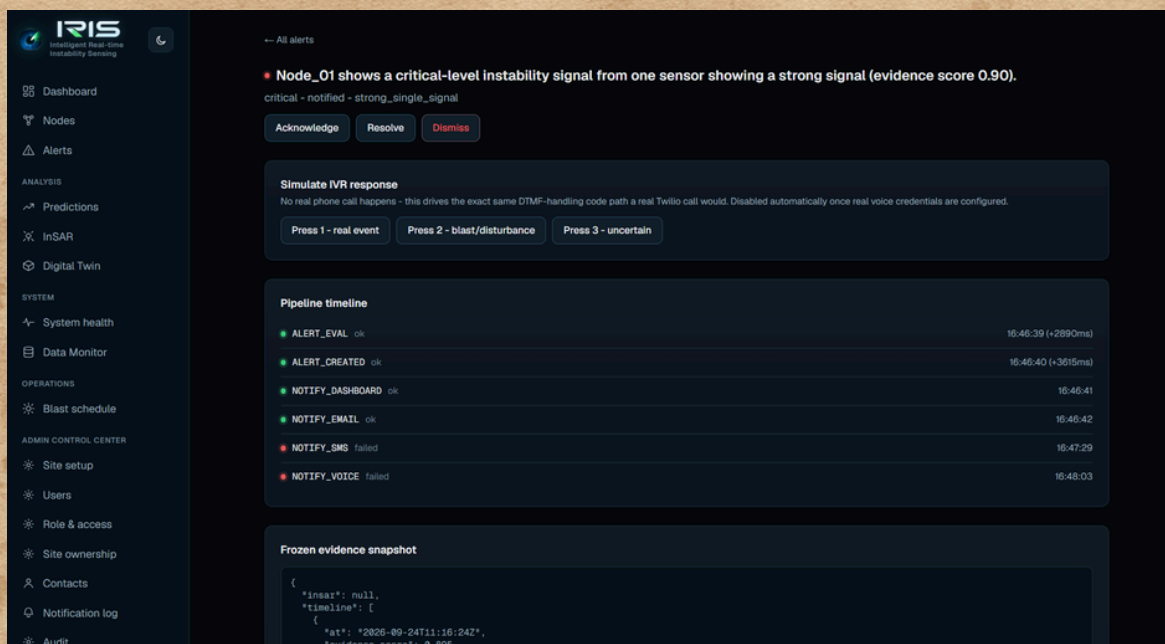


13.7 Feedback -> ML Loop

Every confirmed voice/dashboard verdict is stored in `alert_feedback` and exposed via `GET /api/feedback/export`, giving the ML team labelled ground-truth data captured in the window when a human actually knows the answer – the system's designed mechanism for acquiring real training labels, otherwise scarce for rare failure events.

13.8 What Is Explicitly Out of Scope for AI in This Subsystem

Generative AI may summarize/explain an already-determined event into natural language for the voice call or WhatsApp message. It must never determine whether an event is dangerous, and a conversational/voice-recognition-driven authorization agent is explicitly not built this cycle – DTMF only, for reliability under safety-critical conditions.



13.9 Build Status Flag

Per code audit: Partial – real Twilio/Resend integration exists; the escalation timer runs in-process (not serverless-durable, a known scalability risk); WhatsApp is not implemented. Demo Mode (DEMO_MODE flag) renders the full call UI, script text, and DTMF interaction without touching the telephony provider, with on-screen behavior identical to production – used for reliable recorded demos.

PART 14 — RELIABILITY & NAMING DISCIPLINE

- No single cheap detection stage has permanent veto power; a sufficiently strong single-channel signal can escalate alone.
- Sensor health is a distinct UNKNOWN state, never folded into a false "risk = 0."
- "Fuzzy Risk Index" / "Crack-Ground Anomaly Score," never "probability."
- "LOS displacement," never automatically "vertical subsidence"; relative to a stated reference point, never absolute; "displacement," never "velocity," unless genuinely multi-date-derived.
- Low InSAR coherence means unreliable/unavailable, never zero movement.
- Mock node positions always labelled as mock, never presented as real GNSS measurements.
- PHSR/CPHSR is a baseline susceptibility layer, never directly divergence-compared against a live sensor reading as if they were the same physical unit.
- The cellular voice fallback resolves internet-specific outages, not general network independence.
- The Digital Twin's output is evidence, never presented as ground truth the sensors are checked against.
- No component has yet been empirically validated against real failure data — this remains a well-reasoned design pending validation, not a proven result.

PART 15 — FEASIBILITY, IMPACT & REFERENCES

15.1 Feasibility – Challenge -> Strategy Table

Challenge	Strategy
Real failure data is scarce	Physics-based synthetic pretraining, sim-to-real fine-tuning, rather than assuming abundant labeled data
False positives from routine mining activity	Tiered evidence fusion, event-vs-trend signal distinction (vibration = transient, displacement = trend), spatial coherence requiring corroboration
A faulty sensor could masquerade as a real anomaly	Explicit sensor-health UNKNOWN state, never silently reported as "risk = 0"
A cheap Kalman filter could smooth away a real sudden event	Raw-residual monitoring as an independent second detection pathway
No single mining method fits all of India	Dual-mode architecture directly addressing the real >90% Bord-and-Pillar majority, not just the easier longwall case
Sparse sensor coverage	Geometry-informed (angle-of-draw) placement rather than a uniform grid, plus two independent evidence sources (twin + GNN) compensating for gaps
Unreliable connectivity at remote mine sites	Edge-level local decision-making with zero cloud dependency, dual-rate offline logging, cellular voice fallback for critical alerts
Cost	Commodity ESP32/off-the-shelf sensors vs. commercial industrial wireless geotechnical systems running into lakhs-crores per deployment
Avoiding a real regulatory/engineering trap	Deliberately surface-only sensing (per the PS itself), so no intrinsically-safe/explosion-proof underground hardware certification is required at all

15.2 Impact – Quantified, Real-World-Grounded Points

- Over 90% of Indian underground coal production is Bord-and-Pillar (85.76% mechanised + 0.10% conventional) vs. under 8% longwall (7.63% mechanised + 0.74% conventional) – most generic subsidence-AI proposals implicitly target the minority method; this system targets the real majority.
- 258 underground coal mines recorded in India (FY2021-22) vs. 249 opencast – a large population of sites this kind of system could realistically serve.
- Real validation precedent exists in Indian coalfields: PS-InSAR/SBAS studies over Raniganj (a real room-and-pillar coalfield) have recorded subsidence rates up to -21.18 mm/year, cross-validated against GPS – proof this class of approach already works on real Indian ground.
- Real mechanised depillaring achieves up to 70% extraction in documented Raniganj operations – illustrating the scale of active Bord-and-Pillar mining this system is built to monitor.
- Order-of-magnitude cost reduction: commodity IoT hardware and free satellite data (Sentinel-1/Copernicus) vs. commercial wireless geotechnical monitoring products and paid imagery contracts.
- Real-time vs. satellite-only: Sentinel-1's 6-12 day revisit cycle cannot catch a fast-developing failure; local detection operates on a minute-by-minute cadence.
- Broader impact: proactive risk mitigation for nearby communities, infrastructure, agricultural land, and forests – replacing post-facto damage assessment with genuine early warning.
- Made-in-India, student-prototype-friendly: built entirely on widely accessible hardware (ESP32/Arduino/Raspberry Pi) and free/open data sources.

15.3 Research and References (exact, searchable titles – more precise than topic-only citations)

- Probability Integral Method + Knothe's Time Function – classical, widely published mining subsidence engineering theory; search "probability integral method mining subsidence" / "Knothe time function subsidence."
- CIMFR (Central Institute of Mining and Fuel Research, CSIR) – India-specific 3D subsidence prediction for single- and multi-seam extraction; search CIMFR's own publications/CSIR repository.
- CMRI pillar safety factor formula – India-specific pillar design practice; search "CMRI safety factor formula Indian coal pillar."
- Raniganj InSAR study – SBAS-InSAR subsidence monitoring, Sentinel-1 data, cross-validated with GPS, -21.18 mm/year; search Google Scholar for recent (2023-2024) SBAS-InSAR Raniganj coalfield subsidence papers.
- Surakachhar/Korba InSAR study – "Deformation monitoring of Surakachhar underground coal mines of Korba, India using SAR interferometry"; search on Google Scholar or ResearchGate.
- PHSR/CPHSR – "Subsidence monitoring techniques in coal mining: Indian scenario" and "Assessment of pot-hole subsidence risk for Indian coal mines"; search these exact titles on ScienceDirect/Academia.edu.
- Mechanised depillaring, Raniganj 70% extraction case – "Underground Coal Pillar Extraction Under Shale Roof and Caved Goaf of Upper Lying Seam in Raniganj Coalfields"; search on Academia.edu/ResearchGate.
- Longwall subsidence ratio benchmark – "High-intensity longwall mining-induced ground subsidence in Shendong coalfield, China"; ScienceDirect.
- Tributary Area Theory + Obert-Duvall pillar strength formula – standard mining engineering references; search "Obert-Duvall pillar strength formula tributary area theory."
- Fukuzono's inverse-velocity method – established landslide early-warning literature; search "Fukuzono inverse velocity method failure prediction."
- Government production statistics – Ministry of Coal Annual Report / Coal Controller's Organisation statistics, available on coal.nic.in.
- Google Earth Engine (SoilGrids, OpenLandMap, HWSD, Sentinel-2/Landsat NDVI, SRTM) – earthengine.google.com.
- Sentinel-1 / Copernicus Data Space Ecosystem – dataspace.copernicus.eu.
- ASF HyP3 (on-demand InSAR processing) – hyp3-docs.asf.alaska.edu.
- MintPy (open-source InSAR time-series analysis) – github.com/insarlab/MintPy.